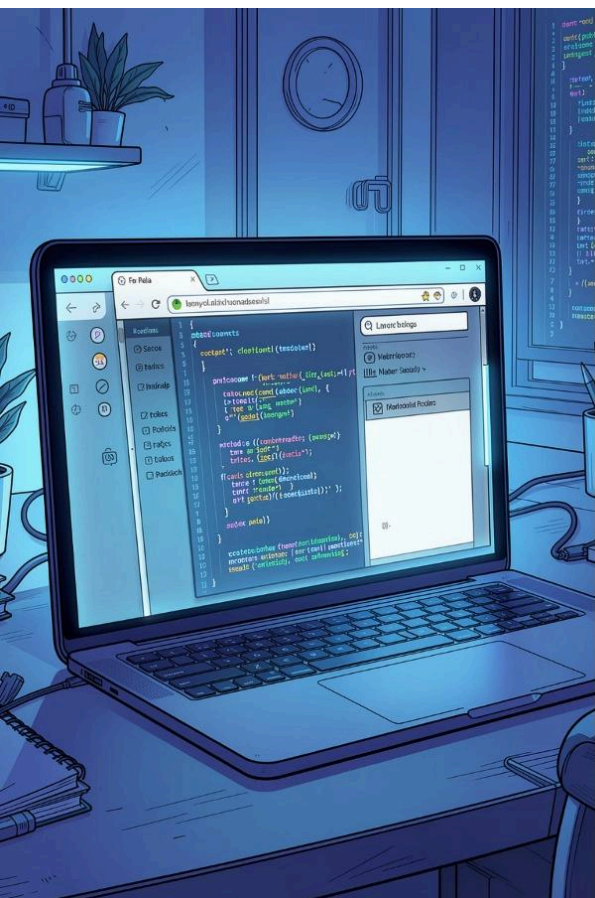




Web Technology (BCS502)

UNIT - 1

A comprehensive introduction to the tools, languages, protocols, and strategies that power the modern web — from the history of the internet to building and deploying real web projects.

What Is Web Technology?

Web technology refers to the tools and techniques used to create and manage websites and web applications. It encompasses languages, frameworks, protocols, and software that make the internet interactive and functional, enabling communication between **clients** (users) and **servers** (data providers).

Web Browsers

Software like Chrome, Firefox, Safari, and Edge that users interact with to access websites.

Web Servers

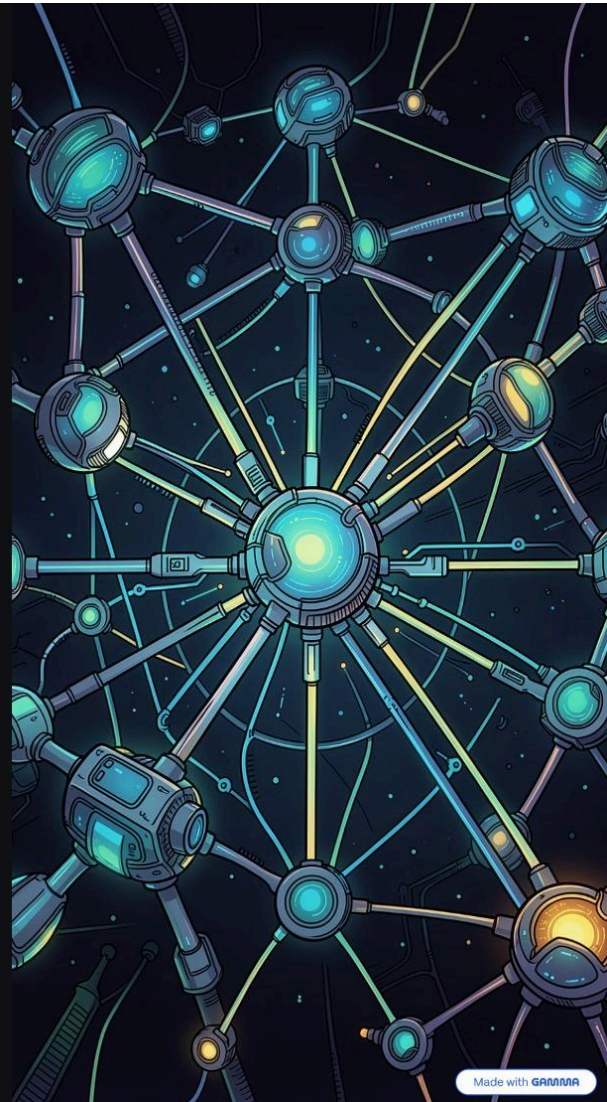
Computers that store, process, and deliver web pages. Examples: Apache, Nginx, Microsoft IIS.

Web Pages

Documents built using HTML, CSS, and JavaScript that define structure, style, and interactivity.

Protocols

Rules defining data transfer over the web. Common ones: HTTP and HTTPS (the secure version).



Made with GAMMA

Web Development Strategies

Web development is broadly divided into **frontend** (client-side) and **backend** (server-side) development, each with specific strategies to ensure websites are effective, efficient, and user-friendly.

Frontend (Client-Side)

- **Responsive Web Design:** CSS media queries, flexible grids for seamless cross-device experience
- **Performance Optimization:** File minification, lazy loading, caching
- **JavaScript Frameworks:** React.js, Vue.js, Angular for dynamic pages
- **Cross-Browser Compatibility:** Test across browsers using tools like BrowserStack

Backend (Server-Side)

- **Microservices Architecture:** Independent services for flexibility and fault tolerance
- **API-First Development:** RESTful APIs for better separation of concerns
- **Database Optimization:** Indexing, caching, normalization
- **Security:** OAuth, JWT, input sanitization against SQL injection, XSS, CSRF
- **Cloud & Serverless:** AWS, Google Cloud for automatic scaling

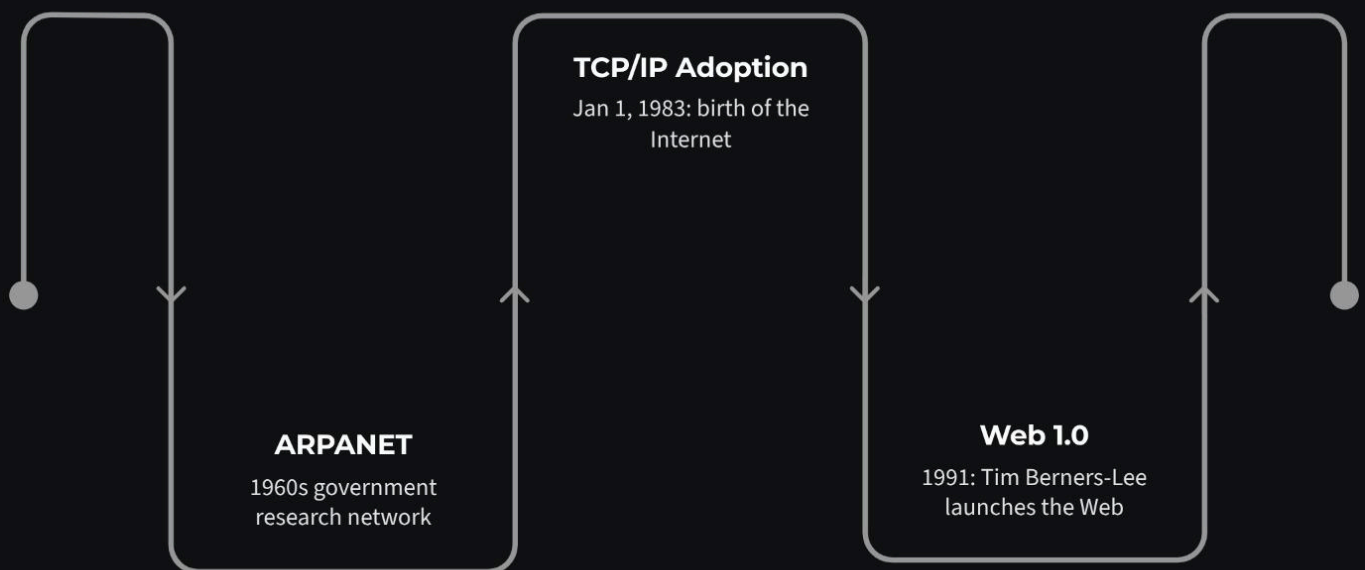
History of the Web & the Internet

The World Wide Web

Invented by **Tim Berners-Lee in 1989** at CERN, the Web went live in 1991, introducing HTML, HTTP, and the first browser. This era, known as **Web 1.0**, featured static websites primarily displaying information with minimal user interaction.

The Internet

The Internet began in the **1960s** as a government research network. The launch of Sputnik spurred the U.S. Defense Department to create **ARPANET**. On **January 1, 1983**, ARPANET adopted **TCP/IP** as its standard — the official birthday of the Internet — enabling all networks to communicate through a universal language.



From ARPANET's government research origins to TCP/IP's universal standard in 1983, and the Web's public launch in 1991 — each phase built the foundation for today's connected world.

Protocols Governing the Web

Protocols are the rules and standards that define how data is transmitted and accessed over the internet.

TCP/IP

TCP breaks data into packets and ensures reliable delivery. **IP** handles addressing and routing to the correct destination.

HTTP / HTTPS

HTTP enables client-server communication on the Web. **HTTPS** adds encryption via TLS/SSL for secure, private communication.

SMTP / POP3 / IMAP

SMTP sends emails. **POP3** downloads and removes emails from the server. **IMAP** keeps emails on the server, synchronizing across devices.

FTP

Transfers files between systems on a client-server model, authenticating with ID and password.

SNMP

Collects management information from network devices (routers, switches, firewalls) for real-time monitoring.

Writing Web Projects: Planning & Setup

Creating a web project follows a structured process from planning to deployment. The first four steps lay the foundation.

01

Define Purpose & Scope

Clarify the goal (blog, e-commerce, portfolio), identify the target audience, and list required features like authentication, database integration, and APIs.

03

Set Up Development Environment

Install a code editor (VS Code), Node.js, and browser dev tools. Initialize a Git repository and install libraries via npm or pip.

02

Plan Project Structure

Create a sitemap (Information Architecture), build wireframes and mockups, and select your technology stack: Frontend (HTML, CSS, JS, React/Vue/Angular), Backend (Node.js, Python, PHP), Database (MySQL, MongoDB), and Version Control (Git/GitHub).

04

Design & Develop Frontend

Build structure with semantic HTML, style with CSS/Sass for responsiveness, add interactivity with JavaScript (ES6+), and test across browsers and devices.

Writing Web Projects: Build, Test & Deploy

Steps 5–10 cover the core development, quality assurance, and launch phases of a web project.

01

Backend Development

Develop server-side logic, design database schemas, set up ORM, and build RESTful APIs for data flow between frontend and backend.

03

Testing & Debugging

Write unit tests, run integration tests, perform end-to-end testing, and use browser dev tools to fix bugs and performance issues.

05

Deploy the Project

Choose a hosting provider (AWS, Google Cloud, Heroku), register a domain, set up CI/CD pipelines (GitHub Actions), and configure SSL/TLS for HTTPS.

02

Authentication & Authorization

Implement sign-up/login with OAuth or JWT. Define user roles and permissions to restrict access to areas like admin panels.

04

Optimize for Performance

Minify CSS/JS, compress images (WebP), implement lazy loading, and use a CDN to serve static files closer to users.

06

Monitor & Maintain

Track performance with Google Analytics and New Relic, log errors via Sentry, apply security patches, and gather user feedback for improvements.

Connecting to the Internet

Connecting to the internet involves establishing a link between a user's device and the larger internet infrastructure through various methods, each with different speeds and reliability.



Wi-Fi

Uses radio waves to transmit data between a router and device. The router connects to a modem, which links to an ISP.



Mobile Data

Uses LTE (4G) and 5G NR protocols. Speed depends on distance from cell towers and network traffic.



Hotspots

A smartphone or dedicated device shares its mobile data via Wi-Fi. Public hotspots are available at cafes and airports.



Dial-up

Older method using standard telephone lines with audible frequency signals to connect to an ISP.



Broadband / DSL / Cable

DSL uses telephone lines at higher frequencies. **Cable** uses coaxial TV cables. Both are always-on and faster than dial-up.

Internet Services & Client-Server Computing

Internet Services

Internet services provide access to vast amounts of information — text, graphics, sound, and software — across four categories:

- **Communication:** Email, Telnet, Newsgroups, IRC, Mailing Lists, VoIP, Instant Messaging
- **Information Retrieval:** FTP, Archie, Gopher, VERONICA
- **Web Services:** Enable applications to exchange information and interact
- **World Wide Web:** Access documents with text, graphics, audio, video, and hyperlinks across servers

Video Conferencing enables two-way video/audio transmission.

Point-to-Point connects two locations; **Multi-point** connects more than two via a Multi-point Control Unit (MCU).

Client-Server Model

In client-server computing, the **server** provides services or resources, and the **client** requests them.

- **Client:** Device (computer, tablet, smartphone) that sends requests — e.g., web browsers, mobile apps, email clients
- **Server:** Computer that processes requests and returns data — e.g., web servers, database servers, file servers
- **Network:** Connects client and server via standard protocols like HTTP or FTP, over a LAN or the internet